

Student Laboratory

Specification Refactoring & Multi-Agent Quality Gates with BMad

Estimated Time: 60-90 Minutes | Deliverable: Worksheet 5 Report

Laboratory Information & Course Context

- **Prerequisites:** Completed Worksheets 1, 2, 3, and 4 (Manual “No-AI” CircleGuard Audit)
 - **Deliverable:** Completed **Worksheet 5: Agentic Integration & Quality Gate Report** (attached in Part 2)
-

1. Laboratory Overview & Objectives

In the previous workshop, you audited the **CircleGuard** pre-implementation specifications without AI assistance. You identified critical defects across the Product Requirements Document (PRD), UX Design Spec, and Architecture Decision Document using the 9-part Specification Weakness Taxonomy

In this laboratory, your role shifts: **You are now the Engineering Leads directing an autonomous AI engineering council powered by BMad.**

Core Learning Objectives

1. **Benchmark** your human audit findings against BMad’s automated review engines (`/bmad-prd validate` and `/bmad-review`).
2. **Orchestrate Multi-Agent Dispute Resolution** using `/bmad-party-mode` to resolve irreconcilable cross-document collisions between Product, Design, and Architecture.
3. **Refactor flawed clauses into formal BMad contracts:**
 - PRD capabilities formulated with verifiable **Given-When-Then (BDD)** criteria.
 - Architecture Decisions formulated as **Architectural Invariants (AD-n)** with strict Status, Binds, Prevents, Rule, and Trade-off clauses.

4. **Execute the Implementation Readiness Gate** (/bmad-sprint-planning) to prove your refactored specifications are ready for autonomous development agents (/bmad-build).

2. Setup & Tools

Verify that your BMad project environment is active with version 6.12.0 and that your installed personas are accessible:

Agent Persona	Module & Role	Core Responsibility
John (bmad-agent-pm)	Product Manager (BMM)	User value, scope control, Jobs-to-be-Done, PRD completeness.
Sally / Freya (wds-agent-freya-ux)	UX Designer (WDS)	Screen states (Loading/Empty/Error/Success), cognitive load, WCAG 2.1 AA.
Winston (bmad-agent-architect)	System Architect (BMM)	Architectural Invariants (AD-n), service boundaries, data ownership, latency trade-offs.
Murat (bmad-tea)	Master Test Architect (TEA)	Testability, risk tiers (P0–P3), edge cases, acceptance test strategy (ATDD).

Always run distinct analytical tasks in a **fresh context window** to maintain maximum reasoning depth and prevent context contamination.

3. Step-by-Step Lab Execution (4 Phases)

Phase 1: Automated Audit & Benchmarking (15 Minutes)

Objective: Compare your human defect log (Worksheets 1 & 2) against BMad’s automated diagnostic tools.

- Open a fresh agent session in your terminal or IDE.
- Run BMad’s automated review engines against the CircleGuard PRD and Architecture exhibits:

```
# Run validation checklist against the PRD
/bmad-prd validate
```

Run multi-lens review (Adversarial & Edge-Case Hunter)
/bmad-review docs/planning-artifacts/prd.md

3. **Analyze the diagnostic output:**
 - What defects did BMad catch that your human triad also found? (*True Positives*)
 - What subtle defects did BMad catch that your team missed? (*AI Discovery*)
 - What nuances did your human team catch that BMad overlooked? (*Human Edge*)
 4. Record your comparative analysis in **Section 5.1 of Worksheet 5**.
-

Phase 2: Resolving Conflicts with Party Mode (15 Minutes)

Objective: Use BMad's Party Mode to convene a live multi-agent roundtable between **John, Winston, Sally, and Murat** to resolve one of the major cross-document collisions you discovered.

1. **Select one of the two core CircleGuard collisions:**
 - **Option A (The Offline Gate Collision - Probe 2):** UX Spec §4.1 mandates offline QR validation in < 200ms, but Architecture page 31 defers offline support to Phase 2.
 - **Option B (The Privacy & Identity Leak - Probe 3):** PRD FR-18 guarantees zero real-name exposure, but UX Spec §4.6 displays real student names on the Professor Circle Dashboard.
2. In a fresh context window, invoke:

/bmad-party-mode

3. **Prompt the agents to debate the trade-offs:** > *"We have a critical cross-document collision in CircleGuard: [Describe your chosen collision verbatim with section numbers]. Winston (Architect), Sally (UX), John (PM), and Murat (Test Architect): Debate the technical, user experience, and risk trade-offs of this contradiction. Do not compromise with vague language. Propose an authoritative engineering resolution that eliminates the collision."*
 4. Observe the debate. Note how each persona defends their constraints (latency vs. user velocity vs. privacy vs. failure modes).
 5. As the Lead Engineer, record the agent arguments and write your final executive resolution in **Section 5.2 of Worksheet 5**.
-

Phase 3: Specification Refactoring into Production Contracts (15 Minutes)

Objective: Transform the consensus reached in Phase 2 into two production-grade BMad engineering artifacts.

1. Architectural Invariant (AD-n)

An architectural invariant is an unbreakable rule that binds future developers and autonomous agents. Follow this mandatory BMad format:

AD-n: [Descriptive Decision Name]

- Status: [ADOPTED]
- Binds: [Specific microservices, mobile components, or repositories constrained]
- Prevents: [Specific architectural divergence, race condition, or regression prevented]
- Rule: [The exact technical requirement written with ZERO elastic language]
- Trade-off: [What is explicitly sacrificed or deferred in exchange for this guarantee]

2. BDD Acceptance Criteria (Given-When-Then)

Rewrite the corresponding PRD requirement into verifiable BDD criteria that a test automation agent (like Murat) can execute directly:

Feature: [Capability name]

Scenario: [Unambiguous scenario title]

Given [precondition: user state, token validity, or network condition]

When [trigger: action executed or event dispatched]

Then [observable postcondition: exact UI state, HTTP response code, or event emitted]

And [secondary condition: data encryption, audit log, or telemetry metric]

6. Author both artifacts in **Section 5.3 of Worksheet 5**.

Phase 4: The Implementation Readiness Gate (15 Minutes)

Objective: Run BMad's pre-implementation quality gate to confirm that the project is ready for code generation.

1. Run the readiness check:

/bmad-sprint-planning

- The agent will execute the check-implementation-readiness routine across the PRD, Architecture, UX, and Epics.
 - Review the terminal status:
 - **PASS** *ightarrow* Specifications are complete, unambiguous, and 100% aligned across documents.
 - **CONCERNS** *ightarrow* Minor gaps identified; can proceed only with explicitly documented risks.
 - **FAIL** *ightarrow* Critical cross-document collisions remain; blocking code implementation.
 - Record your gate result, identified risks, and sign-off in **Section 5.4 of Worksheet 5**.
-

4. Deliverable Submission Instructions

1. Previous Workshop Worksheets 1-4 for validation purposes.
2. Complete all sections of **Worksheet 5** below.
3. Ensure all prompts, agent perspectives, refactored contracts, and terminal outputs are included.
4. Submit the completed document to your course portal before the submission deadline.

Worksheet 5: Agentic Integration & Quality Gate Report

Submission Metadata

Field	Team Entry
Team Name / Triad ID	
Product Lead (Student Name)	
UX / Design Lead (Student Name)	
Architecture Lead (Student Name)	
Submission Date	

5.1 Benchmarking Matrix (Human Audit vs. BMad Automated Review)

Compare the findings from your manual audit (Worksheets 1 & 2) with the findings generated by /bmad-prd validate and /bmad-review:

Audit Dimension	Human Triad Finding (from Worksheets 1 & 2)	BMad Automated Review Finding	Evaluation / Winner
PRD Defects(Design Leaks D, Vagueness V)			☐ Human deeper ☐ BMad deeper ☐ Full agreement

Audit Dimension	Human Triad Finding (from Worksheets 1 & 2)	BMad Automated Review Finding	Evaluation / Winner
UX & UI States <i>(Missing states I, Untestability U)</i>			☐ Human deeper ☐ BMad deeper ☐ Full agreement
Cross-Document Collisions <i>(Capability mismatch Probe X)</i>			☐ Human deeper ☐ BMad deeper ☐ Full agreement
Security & Privacy Gaps <i>(FERPA / HIPAA leaks G)</i>			☐ Human deeper ☐ BMad deeper ☐ Full agreement

Key Takeaway Analysis:

(What critical defects did the AI catch that your human team missed? What subtle domain nuances did your human team catch that the AI missed?)

5.2 Party Mode Resolution Transcript & Executive Decision

- **Chosen Collision:**
 - ☐ **Option A:** Offline Gate Access & Validation (UX §4.1 vs. Architecture page 31)
 - ☐ **Option B:** Professor Dashboard Anonymity vs. Real Names (PRD FR-18 vs. UX §4.6)
- **Target Document Clauses:**
 - Document 1 Clause:

- Document 2 Clause:

Agent Positions Summary (from the /bmad-party-mode debate):

- **John (PM Perspective - User Value & Scope):**

- **Sally / Freya (UX Perspective - Screen States & Friction):**

- **Winston (Architect Perspective - Invariants & Feasibility):**

- **Murat (Test Architect Perspective - Risk & ATDD Testability):**

Final Engineering Resolution (Your verdict as Lead Engineer):

(State the unambiguous engineering decision that permanently eliminates the contradiction)

5.3 Production-Grade Refactored Contracts

Artifact A: BMad Architectural Invariant (AD-n)

AD-____:

-
- Status: [ADOPTED]
 - Binds: _____
 - Prevents: _____

- Rule: _____

- Trade-off: _____

Artifact B: BDD Acceptance Specification (Given-When-Then)

Feature: _____

Scenario: _____
Given _____
And _____
When _____
Then _____
And _____

5.4 Implementation Readiness Quality Gate Sign-Off

- **Command Executed:** /bmad-sprint-planning
- **Resulting Status:**
 - ☐ **PASS** (*Specifications 100% aligned; ready for autonomous code generation*)
 - ☐ **CONCERNS** (*Proceeding with explicitly accepted risks*)
 - ☐ **FAIL** (*Critical blockers remain; code generation blocked*)
- **Remaining Identified Risks (if any):**

Architect Sign-Off Signature: _____

Date: _____

(Declares that specifications are implementation-ready and safe for autonomous agent execution)